

Chapter 32. Analytics Engineering with dbt

Models, sources and refs, layers and materializations, tests, docs, snapshots, incremental models, macros, linting and CI.

Draft chapter

Contents

Chapter 32. Analytics Engineering with dbt

Why this matters

In plain English

32.1 What analytics engineering is

32.2 Your first project

32.3 Models, sources, and `ref`

32.4 Layers: staging, intermediate, marts

32.5 Building the star schema as models

32.6 Materializations: view, table, incremental, ephemeral

32.7 Tests: the part that earns the trust

32.8 Documentation and lineage

32.9 Snapshots: history dbt keeps for you

32.10 Incremental models

32.11 Macros: write a rule once

32.12 Linting and continuous integration

32.13 What dbt is not, and the semantic layer

Common mistakes and how to spot them

In the real world: the two dashboards that disagreed

Tools

The project: a tested mart that feeds a dashboard

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 32. Analytics Engineering with dbt

Part III — Advanced Analytics & Analytics Engineering

Chapter at a glance

You will learn to: explain what analytics engineering is and where dbt fits · set up a dbt project against a real database, with credentials kept out of the code · turn SQL you already write into models, sources, and `refs` that dbt orders for you · lay a project out in staging, intermediate, and marts layers · choose materializations, and know what each one costs · write tests that fail loudly when a business rule breaks, including a reconciliation test · generate documentation and lineage nobody has to maintain by hand · keep type 2 history with snapshots · make a large model incremental and measure what it saves · write a macro so a business rule exists once · lint SQL and run the whole project on every pull request.

Before you start: Chapter 12 and 13 (SQL), Chapter 28 (star schemas, grain, slowly changing dimensions), Chapter 26 (Git and pull requests), Chapter 34 (the command line and environment variables). Chapter 29's habits carry straight over.

Time needed: 16–20 hours, spread over three weeks. Most of it is building the project in section 32.3 onwards.

Tools: dbt Core 1.12 with the Postgres adapter, PostgreSQL 16, Git, and SQLFluff. Everything runs locally; no dbt Cloud account is needed.

Practice data: the `riverstone_2025` database from Chapter 13, plus `riverstone_perf` from Chapter 28 for the incremental example. The finished project is in the companion folder as `riverstone_dbt`, and the chapter builds it from nothing.

Why this matters

Chapter 28 built Riverstone a star schema. It works, and it has four problems that every hand-built warehouse has:

- **Nobody knows the order.** The build script has to run the dimensions before the fact table. That order lives in a file somebody wrote once and in the head of whoever wrote it.
- **Nothing checks it.** If tomorrow's load drops half the customers, the fact table shrinks and the dashboard is quietly wrong.
- **Nobody knows what depends on what.** Change `sales_lines` and you find out what breaks when someone complains.
- **There's no history of changes.** Who changed the revenue rule in March, and why?

Chapter 29 solved exactly these problems for Python: structure, tests, review, version control.

Analytics engineering is the same answer for SQL, and **dbt** is the tool most teams use to do it. You write `SELECT` statements; dbt works out the dependency order, builds the tables and views, runs the

tests, writes the documentation, and does it all from version control with a command that a schedule can run at 3 a.m.

It is also, right now, the job title that sits between analyst and data engineer, and the skill that most often turns "senior analyst" into "analytics engineer" with the salary that comes with it (Chapter 8). Interviewers ask about it directly: "what does *ref* do?", "what's the difference between a table and an incremental model?", "how would you test that a load is complete?"

In plain English

dbt is a kitchen rota for your SQL.

You have thirty dishes to prepare, and some need others finished first: the sauce before the pasta, the stock before the sauce. Writing the order down by hand works until a new dish arrives, and then somebody serves pasta with no sauce.

dbt asks you to write each dish's recipe on its own card, and to say which other cards it uses, by name. From those names it works out the order itself. Add a dish and the rota updates. Change the stock and dbt knows every dish that needs remaking.

Three more things come with the cards:

- **Tasting notes:** rules each dish must satisfy before it leaves the kitchen (no duplicate order ids, no missing revenue, the total matches the source). If a rule fails, the pass stops and somebody is told.
- **A menu:** documentation and a diagram of what feeds what, generated from the same cards, so it can't drift from reality.
- **A logbook:** the cards live in Git, so every change was reviewed, dated, and attributed.

What dbt does not do is move data into the warehouse, or run itself on a schedule. It only transforms what's already there (Chapter 46 handles the moving and the scheduling).

32.1 What analytics engineering is

	Data engineer	Analytics engineer	Data analyst
Works on	pipelines, infrastructure, ingestion	transforming raw tables into trusted models	questions, analysis, dashboards
Main tools	Python, orchestration, cloud	SQL, dbt, Git, the warehouse	SQL, BI tools, spreadsheets
Deliverable	data arrives, reliably	a modelled, tested, documented layer	an answer somebody acts on
Chapter	46, 49	this one	13, 16

The role exists because of one change in how warehouses work. When storage and compute were expensive, you transformed data *before* loading it, because you couldn't afford to keep the raw version: **ETL**, extract-transform-load. Cloud warehouses made storage cheap and compute elastic, so the sensible order became **ELT**: load the raw data first, then transform it inside the warehouse with SQL.

That one change puts transformation where analysts already live, and it's why a tool whose entire job is "run my SQL in the right order, and test it" became a standard.

	ETL	ELT
Transformation happens	in a separate tool, before loading	inside the warehouse, after loading
Raw data kept?	often not	yes, always
Written by	data engineers, often in a GUI or Python	analysts and analytics engineers, in SQL
Fixing a mistake means	reloading from the source	rebuilding models from raw data you still have
dbt's place	not used	the T

32.2 Your first project

Install dbt and the adapter for your database, in a virtual environment (Chapter 29):

```
# terminal
$ uv add dbt-core dbt-postgres          # or: pip install dbt-core dbt-postgres
$ dbt --version
Core:
  - installed: 1.12.5
  - latest:    1.12.5 - Up to date!

Plugins:
  - postgres: 1.11.0 - Up to date!
```

Adapters exist for Snowflake, BigQuery, Databricks, Redshift, DuckDB, SQL Server, and others; the project you write barely changes between them.

Create the project:

```
# terminal, in the folder where you keep your repositories
$ dbt init riverstone_dbt --skip-profile-setup
$ cd riverstone_dbt
$ rm -rf models/example                # the sample models are not worth keeping
```

The folders that matter:

Folder or file	Holds
<code>dbt_project.yml</code>	the project's name and settings: where things build, and how

Folder or file	Holds
profiles.yml	how to connect to the database (which is why it must not contain a password)
models/	your <code>SELECT</code> statements, one file per model
snapshots/	definitions for keeping type 2 history (section 32.9)
tests/	custom tests written as SQL (section 32.7)
macros/	reusable SQL snippets (section 32.11)
seeds/	small CSV files you want in the warehouse (a mapping table, say)
target/	everything dbt generates: compiled SQL, run results, docs. Never in Git

Connecting, without putting a password in a file

profiles.yml tells dbt how to reach the database. This project keeps it beside the code and reads secrets from environment variables, exactly as Chapter 29's package did:

```
# profiles.yml - how dbt connects. Keep the password out of this file and out of Git.
riverstone_dbt:
  target: dev
  outputs:
    dev:                                # the one-year database, for everyday work
      type: postgres
      host: "{{ env_var('RIVERSTONE_HOST', 'localhost') }}"
      port: 5432
      user: "{{ env_var('RIVERSTONE_USER', 'postgres') }}"
      password: "{{ env_var('RIVERSTONE_PASSWORD') }}"
      dbname: riverstone_2025
      schema: dbt_dev
      threads: 4
    large:                              # the three-year database, for the incremental example
      type: postgres
      host: "{{ env_var('RIVERSTONE_HOST', 'localhost') }}"
      port: 5432
      user: "{{ env_var('RIVERSTONE_USER', 'postgres') }}"
      password: "{{ env_var('RIVERSTONE_PASSWORD') }}"
      dbname: riverstone_perf
      schema: dbt_large
      threads: 4
```

Two **targets**: `dev` points at the one-year database, and `large` at Chapter 28's three-year one. The same models run against either, which section 32.10 uses to time an incremental build on 1.9 million rows.

Set the password in your shell (Chapter 34) and check the connection:

```
# terminal, in riverstone_dbt
$ export RIVERSTONE_PASSWORD='your-password'
$ dbt debug --profiles-dir .
Connection test: [OK connection ok]
All checks passed!
```

`--profiles-dir .` tells dbt the profile is in this folder. Without it, dbt looks in `~/.dbt/profiles.yml`, which is where most teams keep it so that credentials never sit next to code at all. Either way, **profiles.yml goes in .gitignore if it contains anything secret**; this one doesn't.

The project file

```
name: 'riverstone_dbt'
version: '1.0.0'
profile: 'riverstone_dbt'

model-paths: ["models"]
seed-paths: ["seeds"]
test-paths: ["tests"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

models:
  riverstone_dbt:
    staging:
      +materialized: view
      +schema: staging
    marts:
      +materialized: table
      +schema: marts

snapshots:
  +target_schema: snapshots
```

Settings apply to a folder and everything under it, and a model can override them. Here, everything in `models/staging` becomes a view in a `staging` schema, and everything in `models/marts` becomes a table in a `marts` schema. Section 32.6 explains why.

32.3 Models, sources, and `ref`

A **model** is a file containing one `SELECT` statement. dbt wraps it in `CREATE TABLE AS` or `CREATE VIEW AS`, using the file name as the object name. That's the whole idea: you write the query, dbt handles the object.

Declaring where raw data comes from

Before building anything, tell dbt what the raw tables are, in `models/staging/sources.yml`:

```

version: 2

sources:
  - name: erp
    description: Riverstone's ERP tables, loaded into the reporting database every night.
    schema: public
    tables:
      - name: orders
        description: One row per order.
        columns:
          - name: order_id
            data_tests: [unique, not_null]
      - name: order_items
        description: One row per product line within an order.
        columns:
          - name: order_item_id
            data_tests: [unique, not_null]
      - name: customers
      - name: products
      - name: employees

  - name: crm
    description: The CRM's nightly customer extract, which holds only current values.
    schema: raw_crm
    tables:
      - name: customers

```

Declaring sources buys three things: `{{ source('erp', 'orders') }}` instead of a hard-coded name, tests that run against the raw tables (the `unique` and `not_null` above), and sources that appear in the lineage graph so everyone can see where the data entered.

Note what isn't there: a `database:` line. Leaving it out lets the same project run against `riverstone_2025` or `riverstone_perf` by switching target. Hard-coding the database is a mistake worth avoiding early, because it fails only when someone tries a second environment.

A first model

`models/staging/stg_orders.sql`:

```

-- One row per order, with names cleaned up and cancelled orders flagged.
select
  order_id,
  customer_id,
  sales_rep_id,
  order_date,
  status,
  status <> 'Cancelled' as is_counted
from {{ source('erp', 'orders') }}

```

And `models/staging/stg_order_items.sql`, which computes net revenue once, for everyone:

```
-- One row per order line, with net revenue and cost computed once, here, for everyone.
select
  oi.order_item_id,
  oi.order_id,
  oi.product_id,
  oi.quantity,
  oi.unit_price,
  oi.discount_pct,
  {{ net_revenue('oi.quantity', 'oi.unit_price', 'oi.discount_pct') }} as net_revenue,
  oi.quantity * p.unit_cost as product_cost
from {{ source('erp', 'order_items') }} as oi
join {{ source('erp', 'products') }} as p on p.product_id = oi.product_id
```

`{{ net_revenue(...) }}` is a **macro**, defined in section 32.11. Ignore it for now: it becomes ordinary SQL.

ref : the function that makes dbt work

Models refer to each other with `{{ ref('model_name') }}`, never by table name. Riverstone's fact model starts:

```
from {{ ref('stg_order_items') }} as oi
join {{ ref('stg_orders') }} as o on o.order_id = oi.order_id
```

`ref` does two jobs at once:

1. **It resolves to the right name at run time.** In your development target it becomes `riverstone_2025.dbt_dev_staging.stg_orders`; in production, the production schema. You never write a schema name in a model.
2. **It builds the dependency graph.** Because `fct_sales_line` refs `stg_orders`, dbt knows `stg_orders` must be built first. Nobody maintains a build order.

See it for yourself. `dbt compile` renders the Jinja without running anything:

```
# terminal, in riverstone_dbt
$ dbt compile --select stg_order_items --profiles-dir .
Compiled node 'stg_order_items' is:
-- One row per order line, with net revenue and cost computed once, here, for everyone.
select
  oi.order_item_id,
  oi.order_id,
  oi.product_id,
  oi.quantity,
  oi.unit_price,
  oi.discount_pct,
  round(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100.0), 2) as net_revenue,
  oi.quantity * p.unit_cost as product_cost
from "riverstone_2025"."public"."order_items" as oi
join "riverstone_2025"."public"."products" as p on p.product_id = oi.product_id
```

The macro has become arithmetic and the sources have become table names. **Everything dbt runs is SQL you can read**, and it's all written to `target/compiled/` when anything surprises you.

32.4 Layers: staging, intermediate, marts

dbt projects converge on the same three layers, because each answers a different question.

Layer	One model per	Job	Materialized as
staging	source table	rename, cast, clean, compute row-level values. No joins that change the grain	view
intermediate	a step in a calculation	the awkward middle of a complicated transformation, split so each piece is readable and testable	view or ephemeral
marts	business concept	the tables people query: facts and dimensions, at a declared grain	table

Riverstone's project after section 32.5:

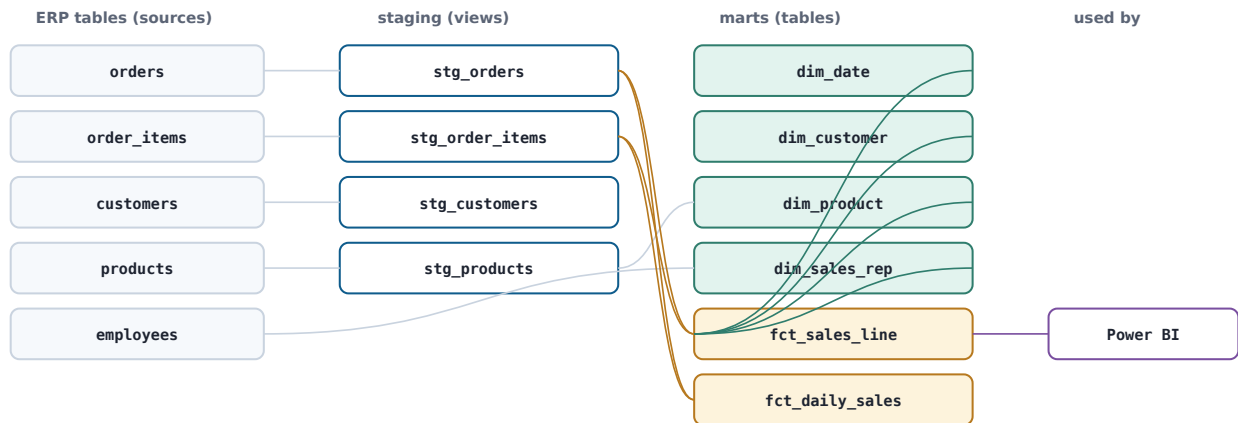
models/	
├─ staging/	
│ └─ sources.yml	where raw data comes from, and its tests
│ └─ schema.yml	tests and descriptions for the staging models
│ └─ stg_customers.sql	
│ └─ stg_order_items.sql	
│ └─ stg_orders.sql	
│ └─ stg_products.sql	
└─ marts/	
│ └─ schema.yml	
│ └─ dim_customer.sql	
│ └─ dim_date.sql	
│ └─ dim_product.sql	
│ └─ dim_sales_rep.sql	
│ └─ fct_daily_sales.sql	
│ └─ fct_sales_line.sql	

Naming conventions matter more than they look: `stg_` for staging, `dim_` and `fct_` for marts, `int_` for intermediate. A reader can tell what a model is from its name, and so can a reviewer looking at a pull request.

Rules of thumb that keep projects readable:

- **One source table, one staging model**, and everything downstream refs the staging model, never the source. If the ERP renames a column, exactly one file changes.
- **Don't join in staging** unless the join can't change the grain. `stg_order_items` joins products only to fetch `unit_cost`, one row per product, which can't fan out (Chapter 28's grain discipline, now enforced by habit).
- **Marts are wide and friendly**. Dimensions carry every attribute a report might slice by, spelled out in business language.
- **If a mart model is getting hard to read, split it** into intermediate models. Views cost nothing to keep.

The graph dbt builds from your `ref()` and `source()` calls



Nobody wrote this order down. dbt reads the `ref()` and `source()` calls in the SQL and builds the graph, then runs it in dependency order, four models at a time. The same graph becomes the lineage diagram in the documentation site.

Figure 32.1 – Sources on the left, marts on the right. dbt derives this graph from the `ref` and `source` calls in the SQL.

32.5 Building the star schema as models

Chapter 28's dimensions become four short models. The date dimension is pure SQL with no source at all:

```

{{ config(materialized='table') }}
-- One row per calendar day of 2025 (Chapter 28's date dimension).
with days as (
    select generate_series(date '2025-01-01', date '2025-12-31', interval '1 day')::date as
full_date
)
select
    to_char(full_date, 'YYYYMMDD')::int as date_key,
    full_date,
    trim(to_char(full_date, 'Day')) as day_name,
    date_trunc('month', full_date)::date as month_start,
    to_char(full_date, 'YYYY-"Q"Q') as quarter_label,
    extract(isodow from full_date) in (6, 7) as is_weekend
from days
  
```

The product dimension is a rename of a staging model:

```

select
    product_id as product_key,
    product_id,
    product_name,
    category,
    list_price,
    unit_cost
from {{ ref('stg_products') }}
  
```

The sales rep dimension adds the "no rep recorded" row that Chapter 28 argued for, so no order is dropped by an inner join:

```

select
    employee_id as sales_rep_key,
    employee_id,
    employee_name as rep_name,
    job_title
from {{ source('erp', 'employees') }}
union all
select -1, null, 'No rep recorded', 'Unknown'

```

The fact table joins the staging models to the dimensions, with the point-in-time join to the customer dimension that Chapter 28 introduced:

```

-- Grain: one non-cancelled order line.
select
    oi.order_item_id,
    oi.order_id,
    to_char(o.order_date, 'YYYYMMDD')::int as date_key,
    c.customer_key,
    p.product_key,
    coalesce(r.sales_rep_key, -1) as sales_rep_key,
    oi.quantity,
    oi.net_revenue,
    oi.product_cost
from {{ ref('stg_order_items') }} as oi
join {{ ref('stg_orders') }} as o on o.order_id = oi.order_id
join {{ ref('dim_customer') }} as c
    on c.customer_id = o.customer_id
    and o.order_date >= c.valid_from::date
    and o.order_date < c.valid_to::date
join {{ ref('dim_product') }} as p on p.product_id = oi.product_id
left join {{ ref('dim_sales_rep') }} as r on r.employee_id = o.sales_rep_id
where o.is_counted

```

`{{ config(materialized='table') }}` inside a model overrides the project default for that model alone; here the project default is already `table` for marts, so the fact model says nothing.

Running it

```

# terminal, in riverstone_dbt
$ dbt build --profiles-dir .
Running with dbt=1.12.5
Found 10 models, 25 data tests, 1 snapshot, 6 sources, 478 macros
Concurrency: 4 threads (target='dev')

1 of 36 START sql view model dbt_dev_staging.stg_customers ..... [RUN]
2 of 36 START sql table model dbt_dev_marts.dim_date ..... [RUN]
...
27 of 36 START sql table model dbt_dev_marts.fct_sales_line ..... [RUN]
30 of 36 PASS assert_fact_total_matches_source ..... [PASS in 0.10s]
...
Finished running 1 incremental model, 1 snapshot, 5 table models, 25 data tests, 4 view models in
1.66 seconds.

Completed successfully

Done. PASS=36 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=36

```

(Trimmed: the full output lists all 36 nodes, and timings vary by machine.)

dbt build is the command to learn. It runs models, tests, snapshots, and seeds together, in dependency order, and **stops a branch of the graph when something upstream fails**, so a failed test doesn't let bad data through to the models below it. `dbt run` builds without testing, and `dbt test` tests without building; `build` is what you want on a schedule.

The result is a schema of tables you can query like any other:

```
SELECT COUNT(*) AS fact_rows,
       ROUND(SUM(net_revenue)) AS net_revenue,
       COUNT(DISTINCT customer_key) AS customer_versions
FROM dbt_dev_marts.fct_sales_line;
```

```
fact_rows | net_revenue | customer_versions
-----+-----+-----
      326 |    4335471 |                23
(1 row)
```

326 lines and ₹4,335,471: the same numbers Chapter 28's hand-built star schema produced, which is the point. Nothing about the warehouse changed. What changed is that the build order, the tests, and the documentation now come from the code itself.

```
SELECT d.quarter_label,
       c.segment,
       ROUND(SUM(f.net_revenue)) AS net_revenue
FROM dbt_dev_marts.fct_sales_line AS f
JOIN dbt_dev_marts.dim_date AS d ON d.date_key = f.date_key
JOIN dbt_dev_marts.dim_customer AS c ON c.customer_key = f.customer_key
GROUP BY d.quarter_label, c.segment
ORDER BY d.quarter_label, c.segment;
```

```
quarter_label | segment | net_revenue
-----+-----+-----
2025-Q1       | Hospitality | 149040
2025-Q1       | Retail      | 394328
2025-Q1       | Wholesale   | 190944
2025-Q2       | Hospitality | 252170
2025-Q2       | Retail      | 198474
2025-Q2       | Wholesale   | 275925
2025-Q3       | Hospitality | 307278
2025-Q3       | Retail      | 333973
2025-Q3       | Wholesale   | 479039
2025-Q4       | Hospitality | 435551
2025-Q4       | Retail      | 562000
2025-Q4       | Wholesale   | 756751
(12 rows)
```

32.6 Materializations: view, table, incremental, ephemeral

A **materialization** is how dbt turns a model's `SELECT` into something in the database. Four cover almost everything.

Materialization	dbt creates	Build cost	Query cost	Use for
view	a view	almost none	the full query, every time	staging models, small transformations
table	a table, rebuilt each run	full rebuild	fast	marts, anything queried often
incremental	a table, updated with new rows	only the new rows	fast	large fact tables (section 32.10)
ephemeral	nothing: the SQL is pasted into the models that ref it	none	none extra	small intermediate steps you don't want cluttering the warehouse

Four materializations, and what each one costs

view CREATE VIEW: nothing is stored	build: free query: the full query, every time	staging models
table CREATE TABLE AS: rebuilt each run	build: full rebuild query: fast	marts
incremental new rows only after the first build	build: 0.90 s vs 13.91 s here query: fast	large fact tables
ephemeral pasted into the models that ref it	build: nothing exists query: recomputed each time	small intermediate steps

Set the default per folder in `dbt_project.yml`; override it in a model with `{{ config(materialized='...') }}`.

Figure 32.2 — The choice is a trade between build cost and query cost.

The defaults in `dbt_project.yml` (views for staging, tables for marts) are the usual starting point, and the reasoning is worth keeping:

- **Staging as views** means no storage and no staleness: a view always reflects the raw table underneath. Since marts are built on top and materialized as tables, the view is read once per run, not once per dashboard.
- **Marts as tables** means the expensive joins happen once per run, not once per person who opens a report.

Switch one and see what happens:

```
# terminal
$ dbt run --select dim_product --profiles-dir .
1 of 1 OK created sql table model dbt_dev_marts.dim_product ..... [SELECT 8 in 0.09s]
```

`SELECT 8` is PostgreSQL telling you eight rows were written. For a view it would say `CREATE VIEW`.

Watch out: a table is a snapshot of the moment it was built. Everything in a mart is as fresh as the last `dbt build`, which is why teams put "data as of" on dashboards (Chapter 28's staleness warning) and why the schedule matters as much as the models. If a number must be live to the second, it doesn't belong in a mart.

32.7 Tests: the part that earns the trust

A dbt **test** is a query that should return **no rows**. If it returns rows, something is wrong, and dbt tells you how many.

Built-in tests

Four generic tests cover most of what goes wrong, and they're declared in YAML beside the models:

```
version: 2

models:
  - name: fct_sales_line
    description: "Grain: one non-cancelled order line. The single source for sales reporting."
    columns:
      - name: order_item_id
        data_tests: [unique, not_null]
      - name: customer_key
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_customer')
                field: customer_key
      - name: date_key
        data_tests:
          - relationships:
              arguments:
                to: ref('dim_date')
                field: date_key
      - name: net_revenue
        data_tests: [not_null]
  - name: dim_customer
    description: "Type 2 customer dimension: one row per customer per period their details stayed the same."
    columns:
      - name: customer_key
        data_tests: [unique, not_null]
      - name: segment
        data_tests:
          - accepted_values:
              arguments:
                values: ['Retail', 'Hospitality', 'Wholesale']
  - name: dim_date
    columns:
      - name: date_key
        data_tests: [unique, not_null]
  - name: fct_daily_sales
    columns:
      - name: order_date
        data_tests: [unique, not_null]
```

- **unique** and **not_null** are grain and completeness checks: exactly Chapter 28's habit of testing what one row represents.
- **relationships** is a foreign key that the warehouse doesn't enforce: every `customer_key` in the fact table must exist in the dimension. This is the test that catches a dimension load that silently dropped rows.

- **accepted_values** catches new categories appearing without warning, which is how "Retail" quietly becomes "retail " and a dashboard loses a segment.

More live in packages: `dbt_utils` adds tests for expressions, row counts between two models, recency, and mutually exclusive ranges; `dbt_expectations` ports the Great Expectations catalog (Chapter 47).

A custom test: the reconciliation

Generic tests check shapes. The test that actually protects the business is the one that checks a number against something outside the model. Chapter 28 reconciled the star schema to the source by hand; here it runs on every build:

```
-- The reconciliation from Chapter 28: the fact table must total what the source says, to the
rupee.
-- A dbt test passes when it returns no rows.
with fact as (select sum(net_revenue) as total from {{ ref('fct_sales_line') }}),
source_total as (
  select sum({{ net_revenue('oi.quantity', 'oi.unit_price', 'oi.discount_pct') }}) as total
  from {{ source('erp', 'order_items') }} as oi
  join {{ source('erp', 'orders') }} as o on o.order_id = oi.order_id
  where o.status <> 'Cancelled'
)
select fact.total as fact_total, source_total.total as source_total
from fact, source_total
where abs(fact.total - source_total.total) > 0.01
```

Any `.sql` file in `tests/` is a **singular test**: it passes when the query returns nothing. This one returns a row only when the fact table and the source disagree by more than a paisa.

What a failure looks like

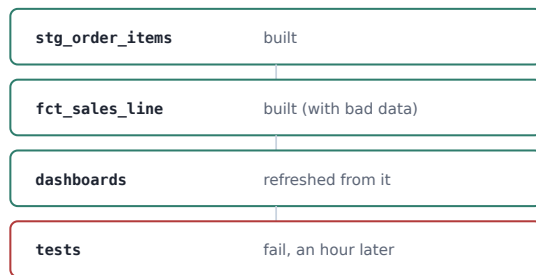
Suppose someone "tidies up" `fct_sales_line` and removes the line that excludes cancelled orders:

```
# terminal
$ dbt build --select fct_sales_line+ --profiles-dir .
2 of 8 FAIL 1 assert_fact_total_matches_source ..... [FAIL 1 in 0.13s]
Completed with 1 error, 0 partial successes, and 0 warnings:
[ERROR]: in test assert_fact_total_matches_source (tests/assert_fact_total_matches_source.sql)
compiled code at target/compiled/riverstone_dbt/tests/assert_fact_total_matches_source.sql
Done. PASS=7 WARN=0 ERROR=1 SKIP=0 NO-OP=0 REUSED=0 TOTAL=8
```

Three things to notice. The build **failed**, so a scheduler sees a non-zero exit code and raises an alarm (Chapter 29's exit codes, Chapter 34's `$?`). The message names the **path to the compiled SQL**, so you can run the failing query yourself and look at the rows. And `fct_sales_line+` built the model *and everything downstream of it*, which is what you want before opening a pull request.

Why dbt build, and not dbt run followed by dbt test

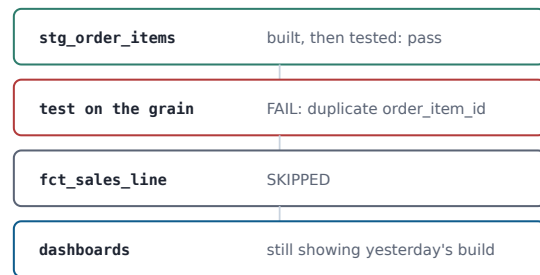
dbt run, then dbt test



Bad numbers reached people before anyone knew.

dbt build runs each model and its tests together, in dependency order, and skips everything downstream of a failure. The run exits non-zero, so the scheduler raises an alarm (Chapter 29's exit codes).

dbt build



The failure stopped the branch. Nothing wrong shipped.

Figure 32.3 – `dbt build` stops a branch of the graph the moment a test fails, so bad data doesn't reach the models below it.

How much to test

Test	Where	Why
<code>unique + not_null</code> on the grain	every model	the single most common failure
<code>relationships</code> on every key in a fact table	marts	catches incomplete dimension loads
<code>accepted_values</code> on categories reports group by	marts	catches new or misspelled values
a reconciliation to the source	one per mart	catches everything else
freshness on sources (<code>dbt source freshness</code>)	sources	catches a load that didn't run

A useful discipline: **when a number is wrong in production, add the test that would have caught it** before you fix the model. The project's test suite then grows out of real failures rather than good intentions.

32.8 Documentation and lineage

Descriptions live in the same YAML as the tests, next to the thing they describe. That's the only way documentation stays true: if it isn't beside the code, it drifts.

```
# terminal
$ dbt docs generate --profiles-dir .
Found 10 models, 25 data tests, 1 snapshot, 6 sources, 478 macros
Building catalog
Catalog written to /home/meera/dbt/riverstone_dbt/target/catalog.json
$ dbt docs serve --profiles-dir .      # opens a site on http://localhost:8080
```

The generated site gives you, for every model: its description, every column with its description and tests, the compiled SQL, and an interactive **lineage graph** showing what it depends on and what depends on it. The graph isn't drawn by anyone; it's the `ref` calls.

What it's actually for, day to day:

- "Where does this number come from?" Click the column, read the description, open the SQL.
- "If I change this source, what breaks?" Look downstream in the graph.
- "Is this metric the same as that one?" Two models both calling `{{ net_revenue(...) }}` are the same by construction; two different formulas are visible as two different formulas.

Two habits make it worth the small effort: describe a model in **one sentence that states its grain** (Chapter 28), and describe any column whose meaning someone could get wrong. Don't describe `order_id` as "the order id".

32.9 Snapshots: history dbt keeps for you

Chapter 28 built a type 2 customer dimension by reconstructing history from an audit log. Most source systems don't keep one: the CRM holds today's values and overwrites yesterday's. A **dbt snapshot** solves that by recording what a table looked like each time it runs.

```
snapshots:
- name: customers_snapshot
  relation: source('crm', 'customers')
  config:
    unique_key: customer_id
    strategy: check
    check_cols: [city, segment]
```

- **strategy: check** compares the listed columns against the stored version and writes a new row when any differ. The alternative, `strategy: timestamp`, uses an `updated_at` column and is cheaper when the source maintains one reliably.
- **unique_key** identifies the business entity, as in Chapter 28.
- dbt adds `dbt_valid_from`, `dbt_valid_to`, and `dbt_scd_id` (a surrogate key, exactly what Chapter 28 built by hand).

The dimension model then reads the snapshot:

```
-- A type 2 customer dimension, built from the snapshot dbt maintains (Chapter 28, section 28.10).
-- dbt records history from the first snapshot onwards, so the first version of each customer is
-- opened far enough in the past to cover the orders we already hold.
with versions as (
    select
        dbt_scd_id as customer_key,
        customer_id,
        customer_name,
        city,
        segment,
        dbt_valid_from,
        dbt_valid_to,
        row_number() over (partition by customer_id order by dbt_valid_from) as version_number
    from {{ ref('customers_snapshot') }}
)
select
    customer_key,
    customer_id,
    customer_name,
    city,
    segment,
    case when version_number = 1 then timestamp '1900-01-01' else dbt_valid_from end as valid_from,
    coalesce(dbt_valid_to, timestamp '9999-12-31') as valid_to,
    dbt_valid_to is null as is_current
from versions
```

Watch it work. Metro Mart moves from Mumbai to Thane, the CRM extract changes, and the next snapshot notices:

```
# terminal
$ psql -d riverstone_2025 -c "update raw_crm.customers set city = 'Thane' where customer_id = 5"
UPDATE 1
$ dbt snapshot --profiles-dir .
1 of 1 OK snapshotting snapshots.customers_snapshot ..... [INSERT 0 1 in
0.22s]
Completed successfully
$ dbt build --select dim_customer+ --profiles-dir .
Completed successfully
Done. PASS=12 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=12
```

Before the change, Metro Mart had one row. After it:

```
SELECT customer_id, customer_name, city,
       valid_from::date AS valid_from, valid_to::date AS valid_to, is_current
FROM dbt_dev_marts.dim_customer
WHERE customer_id = 5
ORDER BY valid_from;
```

customer_id	customer_name	city	valid_from	valid_to	is_current
5	Metro Mart	Mumbai	1900-01-01	2026-09-18	f
5	Metro Mart	Thane	2026-09-18	9999-12-31	t

(2 rows)

Two versions, one closed and one current, built by a tool instead of by the careful `UPDATE` -then-`INSERT` transaction of Chapter 28.

Watch out: a snapshot only knows what it has seen. dbt records history from the first time the snapshot runs, not before. Riverstone's first version therefore starts on the day snapshotting began, which would leave every 2025 order without a matching customer version, so `dim_customer` opens each customer's first version in 1900. Two lessons: **start snapshotting the day you think you might ever want history**, and when a source system does keep an audit log (Chapter 28's `customer_changes`), reconstructing from it gives you history you never captured.

Watch out: snapshots are precious. The snapshot table is the only record of what the source used to say. Never point a snapshot at a different source, never run it against a temporary copy of the data, and back it up like any other irreplaceable table.

32.10 Incremental models

Rebuilding a mart every run is fine until the fact table has millions of rows. An **incremental** model builds fully the first time, and afterwards processes only what's new.

```
{{ config(materialized='incremental', unique_key='order_date',
incremental_strategy='delete+insert') }}
-- One row per day: rebuilt only for days that changed since the last run.
select
    o.order_date,
    count(distinct o.order_id) as orders,
    sum(oi.net_revenue) as net_revenue
from {{ ref('stg_orders') }} as o
join {{ ref('stg_order_items') }} as oi on oi.order_id = o.order_id
where o.is_counted
{% if is_incremental() %}
    and o.order_date >= (select coalesce(max(order_date), date '1900-01-01') from {{ this }})
{% endif %}
group by o.order_date
```

How it works:

- `materialized='incremental'` tells dbt to create the table on the first run, and to insert into it afterwards.
- `is_incremental()` is true only when the table already exists and the run isn't a full refresh. The `where` clause inside it limits the work to recent days.
- `{{ this }}` is the model's own table, so the filter can ask "what's the latest day I already have?".
- `incremental_strategy='delete+insert'` with a `unique_key` replaces the days it rebuilds instead of duplicating them. That matters because the newest day is usually incomplete: it gets rebuilt tomorrow. Other strategies are `append` (fastest, no deduplication) and `merge` (an upsert, on warehouses that support it).

Measured on Chapter 28's three-year database (1,926,847 order lines, 1,096 days), on the author's test machine:

```
# terminal, in riverstone_dbt
$ dbt run --target large --select +fct_daily_sales --full-refresh --profiles-dir .
1 of 3 OK created sql view model dbt_large_staging.stg_order_items ..... [CREATE VIEW in
0.14s]
2 of 3 OK created sql view model dbt_large_staging.stg_orders ..... [CREATE VIEW in
0.13s]
3 of 3 OK created sql incremental model dbt_large_marts.fct_daily_sales ..... [SELECT 1096 in
13.91s]
Finished running 1 incremental model, 2 view models in 14.22 seconds.

$ dbt run --target large --select fct_daily_sales --profiles-dir .
1 of 1 OK created sql incremental model dbt_large_marts.fct_daily_sales ..... [INSERT 0 1 in
0.90s]
Finished running 1 incremental model in 1.05 seconds.
```

13.91 seconds against 0.90: about fifteen times faster, and the gap grows with the table. On a cloud warehouse billed by compute, that ratio is the bill.

When to make a model incremental: when a full rebuild is slow enough to hurt, and rows arrive rather than change. **When not to:** small models (the complexity isn't worth it), and models whose history changes (a customer dimension where old rows get corrected). The rule of thumb is to start with a table and convert when the run time bothers you.

Watch out: incremental models drift. Late-arriving data, back-dated corrections, and changed business logic all leave an incremental table holding rows built by yesterday's rules. Defenses: a **lookback window** (`>= max(order_date) - interval '3 days'`) rather than a hard `>=`, a scheduled **--full-refresh** (weekly is common), and a test that compares the model's totals against a full recomputation. If you change the model's logic, refresh it fully; dbt will not do it for you.

32.11 Macros: write a rule once

A **macro** is a reusable piece of SQL written in **Jinja**, the templating language dbt uses. Riverstone's whole project defines net revenue in one place:

```
{# The one definition of net revenue in the whole project (Chapter 13's rule). #}
{% macro net_revenue(quantity, unit_price, discount_pct) -%}
    round({{ quantity }} * {{ unit_price }} * (1 - {{ discount_pct }} / 100.0), 2)
{% endmacro %}
```

Used in a model as `{{ net_revenue('oi.quantity', 'oi.unit_price', 'oi.discount_pct') }}`, it compiles to ordinary arithmetic (section 32.3). The value isn't saving keystrokes; it's that Chapter 13's argument about whose revenue number is right can only have one answer, and changing the rule means changing one file, reviewed in one pull request.

Jinja gives you three constructs, and they're enough for years:

Syntax	Does	Example
<code>{{ ... }}</code>	output something	<code>{{ ref('stg_orders') }}</code>
<code>{% ... %}</code>	control flow: if, for, set, macro	<code>{% if is_incremental() %}</code>
<code>{# ... #}</code>	a comment that never reaches the database	<code>{# grain: one order line #}</code>

A loop earns its keep when a model repeats itself. Pivoting revenue by category, without typing each category:

```
{% set categories = ['Storage', 'Kitchen', 'Industrial', 'Furniture'] %}

select
    d.month_start,
    {% for category in categories %}
    sum(case when p.category = '{{ category }}' then f.net_revenue else 0 end)
      as revenue_{{ category | lower }}{{ " " if not loop.last }}
    {% endfor %}
from {{ ref('fct_sales_line') }} as f
join {{ ref('dim_product') }} as p on p.product_key = f.product_key
join {{ ref('dim_date') }} as d on d.date_key = f.date_key
group by d.month_start
```

A better version reads the category list from the warehouse itself with `dbt_utils.get_column_values`, so a new category appears in the report the day it appears in the data.

Packages bring other people's macros. `packages.yml` lists them, `dbt deps` installs them:

```
packages:
- package: dbt-labs/dbt_utils
  version: [">=1.3.0", "<2.0.0"]
```

`dbt_utils` is the one nearly every project uses: `date_spine` (Chapter 13's date spine, as a macro), `star`, `surrogate_key`, `pivot`, and a shelf of extra tests. Check the current version on the dbt package hub when you add it.

Watch out: Jinja is a power tool. A model that generates SQL through three nested loops is unreadable, un-lintable, and impossible for the next analyst to debug. The test: could a colleague work out what this produces without running it? If not, write the SQL out.

32.12 Linting and continuous integration

SQLFluff

Chapter 13 promised automatic formatting for teams. **SQLFluff** is the SQL linter and formatter most dbt projects use; it understands dbt's Jinja through a templater, so it lints models rather than choking on `{{ ref(...) }}`.

Configuration lives in `.sqlfluff`:

```
[sqlfluff]
templater = dbt
dialect = postgres
max_line_length = 120

[sqlfluff:templater:dbt]
project_dir = ./
profiles_dir = ./

[sqlfluff:rules:capitalisation.keywords]
capitalisation_policy = lower
```

```
# terminal, in riverstone_dbt
$ pip install sqlfluff sqlfluff-templater-dbt
$ sqlfluff lint models/marts/fct_sales_line.sql
== [models/marts/fct_sales_line.sql] FAIL
L:   2 | P:   1 | ST06 | Select wildcards then simple targets before calculations
      | and aggregates. [structure.column_order]
L:  13 | P:   1 | AM05 | Join clauses should be fully qualified. [ambiguous.join]
L:  13 | P:  38 | ST09 | Joins should list the table referenced earlier first.
      | [structure.join_condition_order]
L:  15 | P:   1 | LT02 | Expected indent of 4 spaces. [layout.indent]
All Finished!
```

`sqlfluff fix` applies the mechanical fixes. Two pieces of advice from teams who have adopted it: **turn rules off rather than arguing about them** (the point is consistency, not any particular style), and **run fix across the project in one commit** when you adopt it, so the first pull request afterwards isn't an unreadable mess of whitespace changes.

Running everything on every pull request

Chapter 26 introduced continuous integration: automated checks on every change. For a dbt project the checks are obvious, and this workflow runs them:

```
# Runs on every pull request: install, lint, then build and test the whole project against a
scratch database.
name: dbt
on: [pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: ${ secrets.CI_DATABASE_PASSWORD }
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready --health-interval 10s --health-timeout 5s --health-retries 5
    env:
      RIVERSTONE_HOST: localhost
      RIVERSTONE_USER: postgres
      RIVERSTONE_PASSWORD: ${ secrets.CI_DATABASE_PASSWORD }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install dbt-core dbt-postgres sqlfluff sqlfluff-templater-dbt
      - run: psql -h localhost -U postgres -c "CREATE DATABASE riverstone_2025" &&
      - run: ./load_test_data.sh
      - run: sqlfluff lint models/
      - run: dbt deps --profiles-dir .
      - run: dbt build --profiles-dir . --target ci
```

What it buys. A pull request that breaks a test cannot be merged unnoticed. A model that doesn't compile is caught before it reaches production. And the reviewer's job shrinks to the part a human is good at: *is this the right business rule?*

Two refinements real teams add:

- **Build only what changed**, with dbt's state comparison: `dbt build --select state:modified+ --state ./prod-artifacts`. On a project with hundreds of models, rebuilding everything on every pull request is too slow.
- **Build into a separate schema per pull request**, so reviewers can query the proposed models without touching production.

Where dbt runs in production

dbt Core is a command-line tool: something has to run it on a schedule. The options, in rough order of how much they cost you:

Option	What it is
<code>cron</code> on a small server (Chapter 34)	the simplest thing that works
An orchestrator: Airflow, Dagster, Prefect (Chapter 46)	dbt as one task among ingestion and downstream jobs
A container on a cloud scheduler	no server to maintain

Option	What it is
dbt Cloud	dbt Labs' hosted product: scheduling, a browser IDE, CI, hosted docs, on a paid plan with a limited free tier

The models, tests, and docs are identical either way. Check current dbt Cloud plans and prices on dbt Labs' site; they change, and this chapter was written against dbt Core 1.12.

32.13 What dbt is not, and the semantic layer

dbt only transforms data that's already in the warehouse. It does **not** extract or load (that's Fivetran, Airbyte, or your own Python, Chapter 46), does **not** schedule itself, is **not** a BI tool, and is **not** a database. A dbt project on a warehouse that nothing loads is a rota for an empty kitchen.

The one boundary that keeps moving is the **semantic layer**: the idea that metric definitions (revenue, active customer, churn) should live in one place that *every* tool asks, rather than being re-implemented in each dashboard. dbt has a semantic layer built on MetricFlow, where you define metrics in YAML alongside models and BI tools query them through an API. Competing approaches exist (Cube, Looker's LookML, and BI tools' own modeling layers), and this area changes fast enough that any specific instruction here would be stale by the time you read it.

The idea, though, is stable, and you already have it: Riverstone's `net_revenue` macro is a one-metric semantic layer. If a metric's definition lives in one file, is tested, and everything downstream refs it, you have most of the benefit. If it lives in six dashboards, no product will save you. Check dbt's documentation for the current state of the semantic layer before adopting it, and Chapter 16 for how BI tools consume a mart.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Hard-coded table names instead of <code>ref</code>	Models build in the wrong order; the lineage graph is wrong	<code>{{ ref(...) }}</code> and <code>{{ source(...) }}</code> , always
<code>database:</code> pinned in <code>sources.yml</code>	Works in dev, fails in every other environment	Let the target decide the database
Credentials in <code>profiles.yml</code> in Git	A warehouse password in the repository forever	Environment variables or a secrets manager
Models that duplicate a business rule	Two dashboards, two revenue numbers	One macro, refed everywhere
Joins in staging that change the grain	Row counts inflate downstream and nobody knows where	Keep staging one-row-per-source-row; join in marts
No tests on the grain	Duplicate keys reach the fact table	<code>unique</code> and <code>not_null</code> on every model's key

Mistake	Symptom	Fix
Tests only on shapes, never on numbers	Everything passes while the total is wrong	A reconciliation test against the source
<code>dbt run</code> on a schedule instead of <code>dbt build</code>	Bad data ships because tests ran after, or not at all	<code>dbt build</code> , which stops the branch on failure
Ignoring a failing test for a few days	The test gets deleted, and the trust with it	Fix or explicitly change the rule, in a reviewed pull request
Everything materialized as a table	Slow builds, warehouse bills	Views for staging, tables for marts, incremental for the big ones
Incremental models never fully refreshed	Old rows built by old logic	<code>Scheduled --full-refresh</code> , and after any logic change
An incremental filter with no lookback	Late-arriving rows never appear	A few days of overlap plus <code>delete+insert</code> on a unique key
Snapshot pointed at a changing source path	History silently lost	Treat snapshot tables as irreplaceable
Documentation written in a wiki	Stale within a month	Descriptions in the YAML beside the model
Jinja loops generating clever SQL	Nobody can debug it, including you in March	Write the SQL out; save Jinja for repetition that is truly mechanical
No CI	Broken models found in production	Build and test on every pull request
One enormous mart model	Impossible to test or review	Split into intermediate models; views are free

In the real world: the two dashboards that disagreed

Anita's dashboard says December revenue was ₹439,824. The finance pack says ₹447,000. The gap is ₹7,176 and nobody can explain it, so both numbers stop being used and the argument is settled by whoever speaks last in the meeting.

Meera spends a morning on it. The dashboard reads Chapter 13's `sales_lines` view, which excludes cancelled orders. The finance pack reads a spreadsheet built from a different export, which includes them. Neither is wrong: they are answers to different questions, and nobody wrote either question down.

The fix isn't a better argument, it's a place for the answer to live. Over two weeks she builds the dbt project in this chapter:

- **`net_revenue` becomes a macro.** One definition, used by staging and by the reconciliation test.
- **`fct_sales_line` states its grain** in its description: one non-cancelled order line. Cancelled orders keep their own model, so the finance question has an answer too, and the two differ by a number anyone can query.
- **The reconciliation test** compares the fact total to the source on every build, so drift is caught the morning it appears, not in a meeting.

- **The lineage graph** shows the dashboard reading `fct_sales_line`, and `fct_sales_line` reading `stg_order_items`, and that reading the ERP's `order_items`. The chain is on a page.
- **Everything runs in CI** on every pull request, so changing the revenue rule is a reviewed decision with a name and a date attached.

Three months later the ERP team renames a column in `order_items`. The nightly build fails at 03:14 with a clear message, one staging model is changed and reviewed the next morning, and nothing downstream needed touching. Before dbt, that rename would have surfaced as a blank dashboard tile at 9 a.m., traced by hand over a day.

What Meera tells Anita: *"The two numbers were answering different questions, and now both questions have a model with its definition written down. The dashboard will keep matching finance, because a test checks it every night and the build fails if it stops."*

Notice that dbt didn't settle the argument. Writing the definition down did. dbt made writing it down the cheapest option, and kept it honest afterwards.

Tools

Versions used for this chapter, checked in September 2026:

- **dbt Core 1.12.5** with **dbt-postgres 1.11.0**, installed with `uv add dbt-core dbt-postgres`. dbt releases often: check `dbt --version` against the documentation, and re-read the release notes before upgrading a project.
- **PostgreSQL 16**, holding `riverstone_2025` and `riverstone_perf`.
- **SQLFluff 4.3.0** with **sqlfluff-templater-dbt** (the templater is a separate package, and forgetting it produces a confusing error).
- **Git** (Chapter 26) and a CI service; the example is GitHub Actions.
- **Adapters** for other warehouses: `dbt-snowflake`, `dbt-bigquery`, `dbt-databricks`, `dbt-redshift`, `dbt-duckdb`, `dbt-sqlserver`. The project changes very little between them; the SQL dialect does.
- **Packages** worth knowing: `dbt_utils` (macros and extra tests), `dbt_expectations` (a larger test catalog), `codegen` (generates boilerplate YAML from your warehouse), `elementary` (observability, Chapter 47).
- **Companion files** in `ch32/`: the complete `riverstone_dbt` project, including its CI workflow and `.sqlfluff`.

The project: a tested mart that feeds a dashboard

Goal: a dbt project someone else could clone, build, and trust, ending in a mart a BI tool reads.

Option A: your own warehouse. Pick one report that matters and rebuild its inputs as a dbt project. Work against a development schema, never production, and keep credentials in environment variables.

Option B: Riverstone. Build the project in this chapter from an empty folder, without copying the companion files, then compare.

Steps:

1. **Set it up:** `dbt init`, a `profiles.yml` reading secrets from the environment, `dbt debug` passing, and the whole thing in Git with `target/` ignored.
2. **Declare your sources**, with `unique` and `not_null` on their keys, and `dbt source freshness` configured for at least one.
3. **Build staging models**, one per source table, materialized as views.
4. **Build the marts:** at least one fact table with a grain stated in its description, and two dimensions.
5. **Test it:** grain tests on every model, `relationships` on every key in the fact table, `accepted_values` on one category, and one **reconciliation test** against the source.
6. **Break something on purpose** and confirm the build fails and the message tells you where.
7. **Add a macro** for one business rule, and use it in at least two places.
8. **Add a snapshot** for a dimension that changes, and prove it captures a change.
9. **Make one model incremental** and time a full refresh against an incremental run.
10. **Document it:** descriptions for every model and for columns whose meaning isn't obvious. Generate the docs and read your own lineage graph.
11. **Add CI** that lints and builds on every pull request, and open one pull request that fails on purpose.
12. **Point a BI tool at the mart** (Chapter 16) and confirm the numbers match your reconciliation.

Deliverables: the repository, the docs site (a screenshot of the lineage graph is fine), the output of a passing `dbt build` and of a failing one, and a half-page note on which tests you chose and why.

Stretch goals:

- Add `dbt_utils` and replace one hand-written test with `expression_is_true` or `equal_rowcount`.
- Add a second target and prove the same models build against both.
- Convert the fact table to incremental with a three-day lookback, and write a test that compares it with a full recomputation.
- Add `dbt build --select state:modified+` to CI so only changed models rebuild.

You've got it when...

- ☐ I can explain ELT, and why transformation moved into the warehouse.
- ☐ I can set up a dbt project whose credentials never appear in the code.
- ☐ Every model of mine uses `ref` and `source`, and I can explain both jobs `ref` does.
- ☐ I lay projects out in staging and marts, and I keep the grain in staging.

- ☐ I choose materializations deliberately and can say what each costs.
 - ☐ My models carry grain tests, relationship tests, and at least one reconciliation.
 - ☐ I run `dbt build`, not `dbt run`, on a schedule, and I know why.
 - ☐ I can read a failing test's output and get to the offending rows.
 - ☐ I write descriptions beside the models and can navigate my own lineage graph.
 - ☐ I can keep type 2 history with a snapshot, and I know what a snapshot can't recover.
 - ☐ I can make a model incremental, with a lookback, and measure what it saves.
 - ☐ I write a business rule once, as a macro, and lint and build on every pull request.
-

Recap

- **Analytics engineering** brings software habits (version control, tests, review, documentation) to SQL transformation. **ELT** put that transformation inside the warehouse, and **dbt** is the tool most teams use for it.
 - A **model** is one `SELECT` in a file. **source** declares raw tables; **ref** connects models, resolves names per environment, and **builds the dependency graph** dbt runs in order.
 - Projects layer into **staging** (one model per source table, views, grain preserved), optional **intermediate**, and **marts** (facts and dimensions, tables).
 - **Materializations**: view, table, incremental, ephemeral. Views for staging, tables for marts, incremental when a rebuild hurts.
 - **Tests** return no rows when they pass. `unique`, `not_null`, `relationships`, and `accepted_values` cover shapes; a **singular test** reconciling to the source covers the numbers. `dbt build` runs models and tests together and stops the branch when one fails.
 - **Documentation** and the **lineage graph** are generated from the YAML and the `ref` calls, so they can't drift.
 - **Snapshots** keep type 2 history from a source that overwrites, but only from the day they start running.
 - **Incremental models** process new rows only: 13.91 seconds against 0.90 on 1.9 million lines here. Use a **lookback window**, a **unique key**, and a scheduled **full refresh**.
 - **Macros** (Jinja) define a business rule once. **Packages** such as `dbt_utils` bring proven macros and tests.
 - **SQLFluff** lints; **CI** builds and tests every pull request; something else (cron, an orchestrator, or dbt Cloud) runs dbt on a schedule.
 - dbt transforms; it does not extract, load, schedule, or visualize.
-

Practice exercises

Work in the `riverstone_dbt` project, against the `dev` target. Predict each answer before you run it.

Warm-up

1. Run `dbt ls` and `dbt ls --select staging`. What do the outputs tell you about how models are addressed?
2. Compile `fct_sales_line` and read `target/compiled/.../fct_sales_line.sql`. Which parts were written by you, and which by dbt?
3. Run `dbt build --select stg_orders+` and explain, from the output, what the `+` did.
4. Query `dbt_dev_marts.fct_daily_sales`. How many rows does it have, and why that number?

Core

5. Add a `stg_employees` staging model, with `unique` and `not_null` tests on `employee_id`, and change `dim_sales_rep` to ref it instead of the source. Does the lineage graph change?
6. Add an `accepted_values` test on `stg_orders.status`, run it, then add a new status to the source data and watch it fail.
7. Write a singular test asserting that no order line has a negative `net_revenue`, and one asserting that every order in `fct_sales_line` has at least one line.
8. Change the `net_revenue` macro to round to zero decimal places. Which models change, and which test catches the consequence?
9. Materialize `dim_date` as a view instead of a table, rebuild, and compare the run output. When would that be the right choice?
10. Add descriptions for every column of `fct_sales_line`, regenerate the docs, and check the column-level lineage.

Stretch

11. Write a model `int_orders_with_totals` as an **ephemeral** materialization, use it in `fct_daily_sales`, and look at the compiled SQL. Where did the model go?
12. Convert `fct_sales_line` to incremental, keyed on `order_item_id`, with a three-day lookback on `order_date`. Then write a test that compares its total against a full recomputation.
13. Add `dbt_utils` to the project and replace the reconciliation test with `dbt_utils.equal_rowcount` plus an expression test. Which version would you rather maintain?
14. Add a `mart_sales_monthly` model using a Jinja loop to pivot revenue by category, then write the same query without Jinja. Which would you put in the repository?
15. Configure `dbt source freshness` on `erp.orders` with a warn-after of 24 hours and an error-after of 48, and make it fail.

Think about it (no code needed)

16. A colleague wants the dashboard to read `stg_order_items` directly, "because it has everything and it's fresher". Give two reasons to say no, and one case where they'd be right.
17. Your project has 300 models and `dbt build` takes 40 minutes. List four things you'd look at, in order.
18. When would you *not* use dbt for a transformation?

Key terms

analytics engineering · ETL · ELT · dbt Core · dbt Cloud · adapter · profile · target · `dbt_project.yml` · `profiles.yml` · model · source · `ref` · `source()` · Jinja · compiled SQL · DAG (dependency graph) · staging layer · intermediate layer · mart · materialization · view · table · incremental model · ephemeral model · `is_incremental()` · `{{ this }}` · incremental strategy · lookback window · full refresh · generic test · singular test · unique · `not_null` · `relationships` · `accepted_values` · source freshness · `dbt build` · `dbt run` · `dbt test` · node selection (`+`, `state:modified`) · snapshot · `dbt_valid_from` · `dbt_valid_to` · `dbt_scd_id` · check strategy · timestamp strategy · macro · package · `dbt_utils` · docs site · lineage graph · SQLFluff · linting · continuous integration · semantic layer · MetricFlow

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 16, Business Intelligence with Power BI**, reads the mart this chapter builds.
 - **Chapter 28, Advanced SQL, Performance & Data Modeling**, is the modeling this chapter automates; its grain and SCD lessons are the reason the tests here look the way they do.
 - **Chapter 46, Pipelines & Orchestration**, gets the data into the warehouse and runs dbt on a schedule alongside everything else.
 - **Chapter 47, Data Quality, Observability & Contracts**, goes further than tests: freshness, anomaly detection, contracts, and who gets paged.
 - **Chapter 49, Storage, Warehouses & Lakehouses**, is where these models run at scale, and where materialization choices become money.
 - **Chapter 62, Data Architecture Patterns**, puts dbt in the wider picture, including when a team shouldn't use it.
 - **Chapter 71, SQL Question Bank**, and **Chapter 77** include dbt and analytics-engineering questions.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. `dbt ls` lists every node in the project: sources, models, snapshots, and tests, one per line, in the form `project.folder.name`. `dbt ls --select staging` lists only what's under `models/staging`. That naming is how **node selection** works everywhere: `--select stg_orders` (one model), `stg_orders+` (it and everything downstream), `+fct_sales_line` (it and everything upstream), `tag:nightly`, `source:erp+`, `state:modified+`. Selection is the skill that makes a 300-model project workable.
2. Everything between `select` and the final `from` is yours. dbt replaced `{{ source('erp', 'order_items') }}` with the fully qualified table name, expanded the `net_revenue` macro into arithmetic, and (for a table materialization) wrapped the result in `create table ... as`. Nothing else

was added: the compiled file is exactly what the database runs, which is why reading it settles most "why is this model doing that?" questions.

3. The `+` selects the model **and everything downstream of it**: `stg_orders`, then the marts that ref it, then the tests on all of them. The output shows the fact models and their tests running after the staging view. Use it before opening a pull request: it proves that a change to one model doesn't break anything below it. `+stg_orders` (the plus in front) would select its upstream instead, which is what you want when a source is misbehaving.

4.

```
SELECT COUNT(*) AS days,
       MIN(order_date) AS first_day,
       MAX(order_date) AS last_day,
       ROUND(SUM(net_revenue)) AS net_revenue
FROM dbt_dev_marts.fct_daily_sales;
```

```
days | first_day | last_day | net_revenue
-----+-----+-----+-----
132 | 2025-01-02 | 2025-12-23 | 4335471
(1 row)
```

One row per day that had at least one non-cancelled order line, which is fewer than 365 because Riverstone doesn't take orders every day (Chapter 13's date-spine problem). The total matches `fct_sales_line`, as it must: both come from the same staging models. If you need a row for every calendar day, join to `dim_date` rather than changing this model.

5. The model is three lines:

```
-- models/staging/stg_employees.sql
select
  employee_id,
  employee_name,
  job_title,
  manager_id
from {{ source('erp', 'employees') }}
```

with tests in `models/staging/schema.yml`, and `dim_sales_rep` changed to `from {{ ref('stg_employees') }}`. The lineage graph gains a node: `erp.employees` → `stg_employees` → `dim_sales_rep` instead of the source feeding the dimension directly. That's the point of the staging layer: one place where the ERP's column names are translated, so a rename upstream is a one-file change.

6.

```
# in models/staging/schema.yml, under stg_orders
- name: status
  data_tests:
    - accepted_values:
        arguments:
          values: ['Delivered', 'Shipped', 'Pending', 'Cancelled']
```

It passes. Now insert an order with status 'Returned' into the source and run `dbt test --select stg_orders`: the test fails with the count of offending rows and the path to the compiled query, which lists the unexpected value. This is the test that catches a source system quietly adding a category, which otherwise shows up as a dashboard whose segments no longer add to the total.

7.

```
-- tests/assert_no_negative_revenue.sql
select order_item_id, net_revenue
from {{ ref('fct_sales_line') }}
where net_revenue < 0
```

```
-- tests/assert_every_order_has_lines.sql
select o.order_id
from {{ ref('stg_orders') }} as o
left join {{ ref('fct_sales_line') }} as f on f.order_id = o.order_id
where o.is_counted and f.order_id is null
```

The second one is worth thinking about before you write it: it will fail for any non-cancelled order whose lines were dropped by a join in the fact model, which is exactly the bug you want to know about, and it's the kind of test that should have caught Chapter 28's point-in-time join problem.

8. Every model that uses the macro changes: `stg_order_items` (and therefore `fct_sales_line` and `fct_daily_sales`), plus the reconciliation test, which uses the same macro and so moves with it. That last part is a trap worth seeing: a test written with the same macro as the model **cannot catch a change to the macro**, because both sides move together. A reconciliation test is stronger when its other side comes from somewhere the macro doesn't touch, such as a figure finance publishes or a hard-coded expected total for a closed period.

9.

```
$ dbt run --select dim_date --profiles-dir .
1 of 1 OK created sql view model dbt_dev_marts.dim_date ..... [CREATE VIEW in
0.07s]
```

The run is faster because nothing is computed, and every query against `dim_date` now runs `generate_series` again. For 365 rows that's free, so a view is a reasonable choice. Make it a table when the dimension is large, when it's joined in every report, or when the warehouse charges for compute rather than storage: then paying once per build beats paying once per query.

10. Descriptions go in `models/marts/schema.yml` under each column. After `dbt docs generate`, the site shows them beside the column, along with the tests attached to it, and the column-level lineage where the adapter supports it. The habit that pays: describe the column's **meaning and unit**, not its name. `net_revenue`: "line value after discount, before tax, in rupees; excludes cancelled orders" is worth writing. "The net revenue" is not.

11. With `{{ config(materialized='ephemeral') }}`, dbt creates nothing in the database. The model's SQL is inlined into every model that refs it, as a CTE at the top of the compiled file. Check `target/compiled/: fct_daily_sales.sql` now starts with `with int_orders_with_totals as (...)`.

Ephemeral models keep intermediate steps out of the warehouse, which is tidy, but they can't be queried, can't be tested on their own, and get recomputed in every model that uses them. Use them for small steps used once or twice.

12. The model gains a config and a filter:

```
{{ config(materialized='incremental', unique_key='order_item_id',
incremental_strategy='delete+insert') }}
...
{% if is_incremental() %}
  where o.order_date >= (select max(order_date) - interval '3 days' from {{ this }})
{% endif %}
```

and the test:

```
-- tests/assert_incremental_matches_full.sql
with incremental_total as (select sum(net_revenue) as total from {{ ref('fct_sales_line') }}),
full_total as (
  select sum({{ net_revenue('oi.quantity', 'oi.unit_price', 'oi.discount_pct') }}) as total
  from {{ source('erp', 'order_items') }} as oi
  join {{ source('erp', 'orders') }} as o on o.order_id = oi.order_id
  where o.status <> 'Cancelled'
)
select * from incremental_total, full_total where abs(incremental_total.total - full_total.total) >
0.01
```

Run it after a few incremental runs, not just after a full refresh: drift is the failure mode you're testing for.

13. `dbt_utils.equal_rowcount` compares two models' row counts, and `dbt_utils.expression_is_true` checks a condition per row. They're shorter to write and consistent across projects. The hand-written reconciliation is more specific: it compares a *total* against the source, to the paisa, which is the thing finance cares about. In practice, keep both: package tests for the mechanical checks, hand-written ones for the business rules that would actually cause an argument.

14. The Jinja version is shorter and adapts when a category is added (more so if the list comes from `dbt_utils.get_column_values`). The written-out version is readable by anyone, lints cleanly, and can be pasted into a query window to debug. For four categories that change once a decade, put the written-out version in the repository. For forty categories that change monthly, use Jinja, and leave a comment saying what it generates.

15. In `sources.yml`:

```
- name: orders
  loaded_at_field: order_date::timestamp
  freshness:
    warn_after: {count: 24, period: hour}
    error_after: {count: 48, period: hour}
```

`dbt source freshness` compares the newest `loaded_at_field` value with now. Against `riverstone_2025`, whose latest order is 31 December 2025, it errors immediately, which is the point of the exercise: **freshness catches the load that didn't run**, which is the single most common cause of a

wrong dashboard, and no model test will ever notice it. In production, point it at a genuine load timestamp rather than a business date.

16. Two reasons to say no: staging models aren't guaranteed stable (they're an implementation detail that gets renamed and restructured), and they don't have the marts' joins, tests, or the grain that reports depend on, so two dashboards reading staging will re-implement the same joins differently. The case where they'd be right: an analyst exploring, in a scratch query, to decide what a mart should contain. Exploration reads whatever it likes; production reports read marts.

17. In order: (1) **Which models are slow?** `target/run_results.json` has a duration per node, and a handful of models usually dominate. (2) **Should the slow ones be incremental?** A full rebuild of a large fact table is the usual culprit. (3) **Are there tables that should be views, or views that should be tables?** Both mistakes cost time. (4) **Is the graph too wide or too deep?** Threads help only if the DAG allows parallelism, and a chain of twelve dependent models can't be parallelized at all. After those, look at warehouse-side things: clustering, partitioning, and the warehouse size (Chapter 49).

18. When the transformation isn't SQL and doesn't want to be: machine-learning features that need Python, or parsing awkward files. When the data isn't in the warehouse yet: dbt can't fetch it. When you need row-by-row processing or true real-time results: dbt runs batches. When the project is one query that one person runs occasionally: a scheduled script is less machinery. And when the team has no version control or review habit, dbt on its own won't create one, though adopting it is a reasonable excuse to start.